

Professional Software Engineering_Lecture_Note

Professional Software Engineering 课堂笔记与翻译

Professional Software Engineering Lecture Notes

Object-Oriented Programming Components

We need to know different components in **object-oriented programming**. We will start with **classes**. What is the usage of a class? Why do we need to use it? For each class, we will have different functions or methods. We will also define what objects are derived from classes. Finally, we will discuss the conditional formatting of all these concepts.

Abstraction in Object-Oriented Programming

One of the key concepts for object-oriented programming is **abstraction**. What does this mean? It is a new concept. The definition is: abstraction is a concept by which concepts are derived from the usage and classification of literal custom. Learning a new concept in programming is best done with an example.

For example, if you often need to multiply a number by itself, a function can abstract this common process. In Python, functions are used for abstracting common processes and building blocks for our code. We also have other abstraction methods that we will see in the next couple of slides.

Function Examples: Round and Absolute Value

Another example is the `round()` function. What is `round()`? It is a function that returns the nearest integer value to a number. For `round(45.6)`, the result is 46. If it is 45.3, it rounds to the lower value, 45. If it is 45.5, it will be 46 or 45 depending on the implementation.

 Student: What is the final result for `round(45.5)`?  Lecturer: Please test it in your code editor or Python command line, as implementations may vary. Some may return 46,

others 45. The general rule is: if the fractional part is lower than 0.5, it rounds to the lower value; from 0.5 up to 0.9, it rounds to the larger value.

For `round(4 + 3)`, which is `round(7)`, the result is 7. These are built-in functions, meaning you do not need to write the function body; they are already available.

Function Parameters and Arguments

Consider the `abs()` function. What is the usage of `abs`? It returns the absolute value. For example, if the input is `-5`, the outcome is `5`. In simple terms, it converts a negative number to positive. What is the input parameter or argument for this function? For `abs(x)`, `x` is the input parameter.

For a function `square(x)`, what is the output? `x * x` is the result or outcome parameter from this function. The input parameter for `square()` could be an expression like `2 + 3 * 5`. This is still the same concept. Instead of passing a variable, we can pass a computed value to the function.

 Student: What is the input parameter for `square(2 + 3 * 5)`?  Lecturer: First, compute `3 * 5 + 2`, which is `17`. So, instead of passing a variable, we are passing the computed value `17` to the function.

专业软件工程讲义笔记

面向对象编程的组成部分

我们需要了解**面向对象编程**中的不同组成部分。我们将从**类**开始。类的用途是什么？为什么我们需要使用它？对于每个类，我们将有不同的函数或方法。我们还将定义从类派生的对象是什么。最后，我们将讨论所有这些概念的条件格式化。

面向对象编程中的抽象

面向对象编程的关键概念之一是**抽象**。这意味着什么？这是一个新概念。定义是：抽象是一种通过使用和字面定制的分类来派生概念的概念。学习编程中的新概念最好通过示例来完成。

例如，如果你经常需要将一个数字乘以自身，函数可以抽象这个常见过程。在 Python 中，函数用于抽象常见过程并为我们的代码构建块。我们还有其他抽象方法，将在接下来的几张幻灯片中看到。

函数示例：四舍五入和绝对值

另一个例子是 `round()` 函数。`round()` 是什么？它是一个返回数字最接近整数值的函数。对于 `round(45.6)`，结果是 46。如果是 45.3，它向下舍入到 45。如果是 45.5，根据实现方式，可能是 46 或 45。

🧑 学生：`round(45.5)` 的最终结果是什么？🧑 讲师：请在代码编辑器或 Python 命令行中测试，因为实现可能不同。有些可能返回 46，其他返回 45。一般规则是：如果小数部分低于 0.5，则向下舍入；从 0.5 到 0.9，则向上舍入。

对于 `round(4 + 3)`，即 `round(7)`，结果是 7。这些是内置函数，意味着你不需要编写函数体；它们已经可用。

函数参数和实参

考虑 `abs()` 函数。`abs` 的用途是什么？它返回绝对值。例如，如果输入是 -5，结果是 5。简单来说，它将负数转换为正数。这个函数的输入参数或实参是什么？对于 `abs(x)`，`x` 是输入参数。

对于函数 `square(x)`，输出是什么？`x * x` 是该函数的结果或输出参数。`square()` 的输入参数可以是一个表达式，如 `2 + 3 * 5`。这仍然是相同的概念。我们可以传递计算值给函数，而不是传递变量。

🧑 学生：`square(2 + 3 * 5)` 的输入参数是什么？🧑 讲师：首先计算 `3 * 5 + 2`，即 17。所以，我们传递计算值 17 给函数，而不是传递变量。

FUNCTION PARAMETERS AND ARGUMENTS

The input parameter for the function `square(2 + 3 * 5)` is the computed value. First, calculate `3 * 5 + 2`, which equals 17. This means we are passing the computed value 17 to the function, not a variable.

🧑 Student: Any question about the parameter and argument? 🧑 Lecturer: Okay, this is a parameter. When we write another function, like `print_message()`, the `message` is a parameter. The specific value we pass to it, like `"Hello"`, is an argument. Make sure to be clear with the distinction between **parameter** (the variable in the function definition) and **argument** (the actual value passed during a function call).

NUMBER OF FUNCTION PARAMETERS AND RETURN VALUES

We want to generate a recipe function. How many input parameters can it have?

👤 Student: Two. 🧑🏫 Lecturer: Here it's two. But generally, how many can we have? As many as we want. Based on the function we are defining, the number of input parameters can vary. Can we leave an input parameter as `null`? Yes, meaning nothing is there. The number is based on the project's needs.

How about the output of a function? How many values can we return? Generally, can we return more than one value?

🧑🏫 Lecturer: Yes, we can return more than one value by using a **data structure**, like a list or an array. This works fine when we are calling the function.

FUNCTION DEFINITIONS ACROSS LANGUAGES

Let's do a quick review of function definitions. If you are familiar with languages like Java, C#, or Python 2, here's a simple comparison.

In Python 3.5 or above, you can optionally define the type of the function's output, similar to C++ or Java, using this form: `def function_name() -> return_type: .` It's not mandatory.

- Usually, we write functions without specifying the return type.
- If you want to specify the type, you use the arrow (`->`) syntax.

What about Java? In Java, you must define the type of the input parameter (e.g., `int`, `String`). In Python, this is not required.

函数参数与实参

函数 `square(2 + 3 * 5)` 的输入参数是计算后的值。首先，计算 `3 * 5 + 2`，结果为 `17`。这意味着我们是将计算值 `17` 传递给函数，而不是一个变量。

👤 学生：关于形参和实参有什么问题吗？ 🧑🏫 讲师：好的，这是一个形参。当我们编写另一个函数，比如 `print_message()` 时，`message` 就是一个形参。我们传递给它的具体值，比如 `"Hello"`，就是一个实参。务必清楚形参（函数定义中的变量）和实参（函数调用时传递的实际值）之间的区别。

函数参数与返回值的数量

我们想创建一个食谱函数。它可以有多少个输入参数？

👤 学生：两个。👤 讲师：这里它是两个。但一般来说，我们可以有多少个？想要多少都可以。根据我们定义的函数，输入参数的数量可以变化。我们可以将输入参数留空为 `null` 吗？可以，这意味着那里什么都没有。数量取决于项目的需求。

函数的输出呢？我们可以返回多少个值？一般来说，我们可以返回多个值吗？

👤 讲师：可以，我们可以通过使用**数据结构**（如列表或数组）来返回多个值。这在调用函数时是可行的。

不同语言中的函数定义

让我们快速回顾一下函数定义。如果你熟悉 Java、C# 或 Python 2 等语言，这里有一个简单的比较。

在 Python 3.5 或更高版本中，你可以选择性地定义函数的返回类型，类似于 C++ 或 Java，使用这种形式：`def 函数名() -> 返回类型:`。这不是强制性的。

- 通常，我们编写函数时不指定返回类型。
- 如果你想指定类型，就使用箭头 (`->`) 语法。

Java 呢？在 Java 中，你必须定义输入参数的类型（例如，`int`、`String`）。在 Python 中，这不是必需的。

SPECIFYING FUNCTION RETURN TYPES AND VARIABLE SCOPE

In Python, to specify the return type of a function, you use the arrow (`->`) syntax. This is optional, unlike in languages like Java where you must declare the types of input parameters (e.g., `int`, `String`).

Local vs. Global Variables

The key difference lies in their scope:

- A **local variable** is defined inside a function's body. It is only valid and accessible within that specific function.

- A **global variable** is defined outside the boundaries of any function. Its value is accessible to all functions defined after it in the code.

🧑 Student: What's the difference between the local variable and global variable? 🧑

Lecturer: If you define a variable inside the body of a function, we call it a local variable. It will be valid and accessible only inside that function. But if you define a variable outside the boundary of the functions, its value will be accessible for the rest of the functions below it.

🧑 Student: What is the usage of the global variable? Can you give an example? 🧑

Lecturer: For example, a constant like `g = 9.8` (gravity). If we don't want to repeat this value inside every function that needs it, we define it as a global variable instead of defining it repeatedly. This is one application; we will see more.

When to Use Functions

As a software developer, you should consider defining a function in these cases:

- To break down a large project or problem into smaller, more manageable sections.
- To assign responsibility for one specific part of the larger problem to a single function.

It's not a strict rule, but this modular approach is common.

Function Size and Responsibility

There is no strict limit on the number of lines of syntax inside a function. However, when defining multiple functions, you should avoid writing too many lines in a single function. A function should be responsible for one coherent part of the project. If you combine different parts of the project into one function, it defeats the purpose of modularization.

指定函数返回类型与变量作用域

在 Python 中，要指定函数的返回类型，需使用箭头 (`->`) 语法。这是可选的，不像在 Java 等语言中必须声明输入参数的类型（例如 `int`、`String`）。

局部变量与全局变量

关键区别在于它们的作用域：

- **局部变量** 在函数体内定义。它仅在该特定函数内有效且可访问。
- **全局变量** 在任何函数边界之外定义。其值可被其后定义的所有函数访问。

🧑 学生：局部变量和全局变量有什么区别？🧑 讲师：如果你在函数体内定义一个变量，我们称之为局部变量。它只在该函数内部有效且可访问。但如果你在函数边界之外定义一个变量，它的值可以被其下方其余的函数访问。

🧑 学生：全局变量的用途是什么？能举个例子吗？🧑 讲师：例如，像 `g = 9.8`（重力加速度）这样的常量。如果我们不想在每个需要它的函数内部重复这个值，就将其定义为全局变量，而不是重复定义。这是一个应用场景；我们还会看到更多。

何时使用函数

作为软件开发人员，在以下情况应考虑定义函数：

- 将大型项目或问题分解为更小、更易管理的部分。
- 将大型问题中某个特定部分的责任分配给单个函数。

这不是严格的规则，但这种模块化方法很常见。

函数规模与职责

函数内部的语法行数没有严格限制。然而，在定义多个函数时，应避免在单个函数中编写过多行代码。一个函数应负责项目中一个连贯的部分。如果将项目的不同部分组合到一个函数中，就违背了模块化的目的。

FUNCTION DESIGN PRINCIPLES

We don't care about the exact number of lines in a function. However, when defining different functions, you shouldn't write too many lines of syntax inside a single function. This is because a function should belong to one specific part of the project or problem. Combining different parts of the project into one function defeats the purpose of modularity.

Mostly, you should write functions in a way that allows you to reuse them for other projects or problems. This is why you need to think about how to generalize the function. Consequently, you will not combine everything together and write more than 40 lines. Even nowadays, the trend is to define smaller, more focused functions.

Reviewing Function Syntax

Please check this slide and read it quickly if you haven't done that.

 Lecturer: What is this `error` equal to? What is the name of the function here, just under zero? What is the input parameter? What is this `error` because you wrote it `error` `= ...`?

Whatever this function is going to return will be saved in this variable. It makes sense: you are calling the function, passing the parameter, and you expect to have some result. Whatever the result is, it will be stored in the variable.

 Lecturer: How about here? What is the name of the function? `screen_table`. We have two input parameters, names are marked. Then we have another function where we're going to pass the data. You see the different uses of the function there.

Okay, now we understand abstraction and functions. Any questions before starting the definition of the class?

INTRODUCTION TO CLASSES

Now, what is a class? What's the difference between a class and a function? In a class, you can define variables, member functions, and behavior. This means inside the class, we can define different methods that belong to that class and are accessible through it.

 Lecturer: How? By using an object to have access to those methods or variables we define.

It means we want to make a block, or a couple of different blocks, in our project based on different classes. Each class is going to have a specific boundary and responsibility. Responsibility means the methods: what are the methods available?

 Lecturer: Can we define a class without an object? Yes, but what is the usage of that? At the end of the day, you can think of defining a class like a chef cooking something in a pot. Do you need any spoon to have access inside the pot? Definitely yes. That spoon, which you are going to use, is like an instance of that class.

It means the tools, or variables, needed to be able to have access inside the class. Otherwise, you define a class with lots of variables and methods, but how can you access them? By using an instance variable. Here we call it an object or an instance of the class. Careful about that. We are saying that everything in Python is an object.

Let's do a simple activity to make sure we understand the concept behind the class. But first, what is the format?

函数设计原则

我们并不关心函数的确切行数。然而，在定义不同的函数时，你不应该在一个函数内部编写过多的语法行。这是因为一个函数应该属于项目或问题的某个特定部分。将项目的不同部分组合到一个函数中，就违背了模块化的目的。

大多数情况下，你应该以可以复用于其他项目或解决问题的方式来编写函数。这就是为什么你需要思考如何泛化这个函数。因此，你不会把所有东西都组合在一起并编写超过40行代码。即使在今天，趋势也是定义更小、更聚焦的函数。

回顾函数语法

请检查这张幻灯片，如果还没看的话快速阅读一下。

👤 讲师：这个 `error` 等于什么？这里的函数名是什么，就在零下面？输入参数是什么？这个 `error` 是什么，因为你写了 `error = ...`？

无论这个函数将返回什么，都会被保存到这个变量中。这很合理：你调用函数，传递参数，并期望得到某个结果。无论结果是什么，它都将存储在这个变量中。

👤 讲师：这里呢？函数名是什么？`screen_table`。我们有两个输入参数，名称已标记。然后我们有另一个函数，我们将向其中传递数据。你在那里看到了函数的不同用法。

好的，现在我们理解了抽象和函数。在开始定义类之前有什么问题吗？

类简介

那么，什么是类？类和函数有什么区别？在类中，你可以定义变量、成员函数和行为。这意味着在类内部，我们可以定义属于该类并通过该类访问的不同方法。

👤 讲师：如何访问？通过使用对象来访问我们定义的那些方法或变量。

这意味着我们希望在项目中基于不同的类创建一个或多个代码块。每个类都将有一个特定的边界和责任。责任指的是方法：有哪些可用的方法？

👨‍🏫 讲师：我们可以定义一个没有对象的类吗？可以，但那有什么用呢？归根结底，你可以把定义类想象成厨师在锅里烹饪东西。你需要勺子来接触锅里的东西吗？当然需要。你要用的那把勺子，就像是那个类的一个实例。

这意味着需要工具或变量才能访问类内部的内容。否则，你定义了一个包含许多变量和方法的类，但如何访问它们呢？通过使用实例变量。这里我们称之为对象或类的实例。请注意这一点。我们说Python中的一切都是对象。

让我们做一个简单的活动，以确保我们理解类背后的概念。但首先，格式是什么？

DEFINING A CLASS: SYNTAX AND KEYWORDS

We need to understand the format for defining a class. First, recall the keyword to define a function.

👨‍🎓 Student: What is the keyword to define the function? 👨‍🏫 Lecturer: `def` is the keyword. Then comes the function name, followed by input parameters.

Similarly, `class` is the keyword for defining a class. The compiler understands this keyword. The basic format is `class` followed by the class name.

```
class ClassName:
    # Body of the class
```

Comparing Function and Class Syntax

Going back to a function: it has a name and possibly input parameters. For a class in its basic format, we do not have input parameters. We cannot imagine having input parameters for a class at this basic level. For advanced levels, we will see how to define input parameters for a class if needed.

For now, the structure is: the class name, and then inside the body of that class, you can define different methods—as many as you want based on the project.

Analyzing a Code Example

Look at this example. We have a function named `display`. Do not get confused; this function is not inside a class. It is just a simple function with one input parameter. Inside the

function body, we have one line of syntax.

🧑‍🎓 Student: What is this line? 🧑‍🏫 Lecturer: You call `color`. `color` is an object, an instance of that class. This is the basic format.

Okay, we will leave the detailed initiation for later. Let's do a simple activity involving a bit of coding. We will use class functions, and touch on local and global variables to refresh our memory if we've forgotten. Then we will go into more detail.

Addressing Background Knowledge

This should be straightforward for those who have worked with object-oriented programming in other languages. That's why we want to review, to ensure you haven't missed any concepts or if there is a gap from your previous studies or work.

For those who are new: based on your development work from yesterday, I checked a couple of submissions. Some received a thumbs up, some did not. A thumbs up means it's acceptable. For the rest, you missed the **main function**.

Project Structure in Python

From now on, whenever we develop a project or activity, we should write a full project. This means we should have a main function in the Python format. I will show it to you. It is not a `main` function like in C++, but we should follow a specific format in Python.

定义类：语法与关键字

我们需要理解定义类的格式。首先，回忆定义函数的关键字。

🧑‍🎓 学生：定义函数的关键字是什么？ 🧑‍🏫 讲师： `def` 是关键字。然后是函数名，后面跟着输入参数。

类似地， `class` 是定义类的关键字。编译器能理解这个关键字。基本格式是 `class` 后跟类名。

```
class 类名：  
    # 类的主体
```

比较函数与类的语法

回到函数：它有名称，可能还有输入参数。对于类的基本格式，我们没有输入参数。在这个基础层面，我们无法想象类有输入参数。在高级阶段，我们将看到如何在需要时为类定义输入参数。

目前，结构是：类名，然后在类的主体内部，你可以定义不同的方法——根据项目需要，想定义多少就定义多少。

分析代码示例

看这个例子。我们有一个名为 `display` 的函数。不要混淆；这个函数不在类内部。它只是一个带有一个输入参数的简单函数。在函数主体内部，我们有一行语法。

👨‍🎓 学生：这一行是什么？👨‍🏫 讲师：你调用 `color`。`color` 是一个对象，是那个类的一个实例。这是基本格式。

好的，我们将把详细的初始化留到以后。让我们做一个简单的活动，涉及一点编码。我们将使用类函数，并涉及局部变量和全局变量，以便在忘记时刷新记忆。然后我们将更详细地讲解。

针对背景知识

对于在其他语言中使用过面向对象编程的人来说，这应该是很直接的。这就是为什么我们要回顾一下，以确保你没有遗漏任何概念，或者你的先前学习或工作中是否存在知识缺口。

对于新手：根据你们昨天的开发作业，我检查了几份提交。有些得到了点赞，有些没有。点赞意味着可以接受。其余的，你们漏掉了 **主函数**。

Python 中的项目结构

从现在开始，每当我们开发项目或进行活动时，我们都应该编写完整的项目。这意味着我们应该有一个符合 Python 格式的主函数。我会展示给你们看。它不像 C++ 中的 `main` 函数，但我们应该遵循 Python 中的特定格式。

PROJECT DEVELOPMENT GUIDELINES

Based on your development assignments from yesterday, I checked several submissions. Some received a thumbs-up, some did not. A thumbs-up means it's acceptable. The rest missed the **main function**.

Project Structure in Python

From now on, whenever we develop a project or conduct an activity, we should write a complete project. This means we should have a main function in the Python format. I will show it to you. It's not a `main` function like in C++, but we should follow a specific format in Python.

 Student: Is it the industry standard to use type casting? In newer versions of Python, we can actually use type hints. Is the Python community pushing us to customize types? Is that the standard?  Lecturer: For this first step of learning, I don't want to push and jump to that level at the moment. But definitely yes, we will reach that side soon. I don't want you to feel too much at the beginning and miss some part as a gap. That's why we want to follow the regular and standard pathway, and then we will automatically use type casting.

FIRST CONNECTED PROJECT ACTIVITY

Let's check your tasks. We want to develop the first basic connected project using **one class** and a minimum of **two functions**.

Project Requirements

- We want to see the flexibility of using **local and global variables** based on your project's needs during development.
- The goal is to sort student data based on age.
- **Number of Students:** Minimum of 3, maximum is up to you.
- **Data Structure:** You can choose to use a **DataFrame** or a **list**.
- **Data Fields:** The data structure (list or other) must contain:
 1. Student Name
 2. Student ID
 3. Age
- **Functionality:** Compare and sort students from youngest to oldest, then print the sorted list of student names and ages.

Implementation Notes

- Use the standard style for the `main` function in your Python coding.
- Define one class and two or three functions. For example:

- One function for input (asking for name, student ID, age).
 - Another function for sorting.
 - A function or method for printing the table. You can decide whether to print within the main function or elsewhere.
- Try to use a **global variable** if needed or if you can appropriately integrate it into your project design. It is not mandatory but is a good practice to understand its usage.
 - I will work on it simultaneously while you are coding.

To manage our time, I have set a timer. Let's review one possible structure for this activity together. I will open the magnifier to make it easier to read, especially at the end of the class.

完整中文翻译

项目开发指南

根据你们昨天的开发作业，我检查了几份提交。有些得到了点赞，有些没有。点赞意味着可以接受。其余的，你们漏掉了 **主函数**。

Python 中的项目结构

从现在开始，每当我们开发项目或进行活动时，我们都应该编写完整的项目。这意味着我们应该有一个符合 Python 格式的主函数。我会展示给你们看。它不像 C++ 中的 `main` 函数，但我们应该遵循 Python 中的特定格式。

 学生：使用类型转换是行业标准吗？在 Python 的新版本中，我们实际上可以使用类型提示。Python 社区是在推动我们自定义类型吗？那是标准做法吗？ 讲师：对于学习的第一步，我目前不想推动并跳到那个水平。但肯定是的，我们很快就会接触到那方面。我不想让你们在开始时感到压力太大，从而遗漏某些部分造成知识缺口。这就是为什么我们要遵循常规和标准的路径，然后我们自然会使用类型转换。

第一个连接项目活动

让我们检查一下你们的任务。我们要使用一个类和至少两个函数来开发第一个基础的连接项目。

项目要求

- 我们希望看到根据开发过程中项目的需求，灵活使用**局部变量**和**全局变量**。
- 目标是根据年龄对学生数据进行排序。
- **学生数量**：最少 3 个，最多由你决定。
- **数据结构**：你可以选择使用 **DataFrame** 或列表。
- **数据字段**：数据结构（列表或其他）必须包含：
 1. 学生姓名
 2. 学生 ID
 3. 年龄
- **功能**：比较并按从最年轻到最年长的顺序对学生进行排序，然后打印排序后的学生姓名和年龄列表。

实现说明

- 在 Python 编码中使用 `main` 函数的标准风格。
- 定义一个类和两到三个函数。例如：
 - 一个用于输入的函数（询问姓名、学生 ID、年龄）。
 - 另一个用于排序的函数。
 - 一个用于打印表格的函数或方法。你可以决定是在主函数内部打印还是在其他地方打印。
- 如果需要的话，或者如果你能恰当地将其整合到你的项目设计中，请尝试使用**全局变量**。这不是强制性的，但这是理解其用法的良好实践。
- 我将在你们编码的同时进行开发。

为了管理我们的时间，我设置了一个计时器。让我们一起回顾一下这个活动的一种可能结构。我将打开放大镜，以便更容易阅读，尤其是在课程结束时。

Activity Structure and Coding Guidelines

Let's review the structure for this activity together.

PROJECT REQUIREMENTS

- **Global Variable**: It is possible to use a **global variable**. It's not mandatory, but it's good practice to see how we can use at least one. For example, I have defined a `student_list`. This variable is defined outside the boundary of any classes or functions, on the first line.
- **Class**: Use a class. The name is optional; choose whatever is meaningful for your project, like `Student`.

- **Functions:** Have a minimum of two functions inside your project. One function is for collecting input information, and another is for possibly showing the sorted data.

MAIN FUNCTION AND COMPILER DIRECTIVE

At the bottom, we will have the `main` function. The `if __name__ == "__main__":` line is a keyword for the compiler. You cannot change or remove these underscores. If this directive is present, the Python compiler will start executing from this point.

For the first step, we are going to call the first function from within `main`.

CLASS AND FUNCTION IMPLEMENTATION

Let's look inside the class. I will try not to use the initializer (`__init__`) as we will learn about it later. Our first variable (the list) is empty initially.

Inside the `Student` class, we have methods like `set_data` and `display_information`. Inside a class, you can define one or more functions.

The `set_data` function (with `self` as a parameter) allows you to set data based on the function's name. We will set initial values for variables like `name`, `age`, and `student_id`. For now, setting these initial values in this way is mandatory, as we will update the method to use an initializer later.

CODING STYLE FLEXIBILITY

You might write this in another way, and it could still be correct. We shouldn't all follow exactly the same style; the style of coding can vary.

完整中文翻译

活动结构与编码指南

让我们一起回顾本次活动的结构。

项目要求

- **全局变量**：可以使用**全局变量**。这不是强制性的，但这是了解如何使用它的良好实践，至少使用一个。例如，我定义了一个 `student_list`。这个变量定义在任何类或函数的边界之外，在第一行。
- **类**：使用一个类。类名是可选的；选择对你的项目有意义的名称，例如 `Student`。
- **函数**：在你的项目中至少包含两个函数。一个函数用于收集输入信息，另一个可能用于显示排序后的数据。

主函数与编译器指令

在底部，我们将有 `main` 函数。`if __name__ == "__main__":` 这一行是编译器的关键字。你不能更改或删除这些下划线。如果存在这个指令，Python 编译器将从此处开始执行。

第一步，我们将在 `main` 中调用第一个函数。

类与函数实现

让我们看看类内部。我将尽量不使用初始化器（`__init__`），因为我们稍后会学习它。我们的第一个变量（列表）最初是空的。

在 `Student` 类内部，我们有像 `set_data` 和 `display_information` 这样的方法。在一个类内部，你可以定义一个或多个函数。

`set_data` 函数（以 `self` 作为参数）允许你根据函数名称设置数据。我们将为 `name`、`age` 和 `student_id` 等变量设置初始值。目前，以这种方式设置这些初始值是必需的，因为我们稍后将更新此方法以使用初始化器。

编码风格的灵活性

你可能以另一种方式编写，它可能仍然是正确的。我们不应该都遵循完全相同的风格；编码风格可以有所不同。

SETTING INITIAL VALUES AND FUNCTION PURPOSE

We are going to set the initial value for this asset. Is it mandatory? Yes, for now, because later on we will update it based on the initializer. For now, we will leave it as a function to set the initial value.

You might write it in another way; it could still be correct. We shouldn't all follow exactly the same style; the style of coding can vary, but all implementations should be correct.

THE `DISPLAY_INFO` FUNCTION

We have another function to display information. What are we going to display as an outcome? Just two values: one is **name**, another one is **age**. We don't care about the student ID for the output. That's why we are going to just return the name and age at the point where we call the `display_info` function.

CALLING FUNCTIONS FROM `MAIN`

Through the `main` function (written around line 55), the first function we need to call is `collect_students`. For the Python compiler, we can start calling this function. Let's jump to this function.

INSIDE THE `COLLECT_STUDENTS` FUNCTION

From line 55, we go to line 20. Here at line 20, we have the `collect_students` function. Let's open this function to see inside. The name is clear: we want to collect the input data for how many students. If you mention three, you can change it to as many as you want. You can even ask the end user how many students they want to enter.

UNDERSTANDING THE `FOR` LOOP

This `for` loop is a little bit different from loops in C++ or C programming, or even MATLAB or Java. This is a Python-style `for` loop. `i` is a counter for the loop. It starts automatically by default from zero; it's an index (0, 1, 2). In the `range(3)`, it means 0, 1, and 2 will be considered, but 3 will not.

We are asking to print "student" because it's the first student, second student, third student. That's why we use `i+1`. We don't have a student 0; we have student number 1, student number 2, student number 3. That's why we want to print it out. Then we ask the end user: "What is the name?" You remember yesterday we used the `input` function to ask for the name of the student, and the value will be stored. All these three values (name, ID, age) are collected inside the loop because we want to ask for them multiple times.

VARIABLE SCOPE AND OBJECT CREATION

 Student: Is this a global variable or a local variable?  Lecturer: It's a local variable because it's inside the function. All of these variables are local.

When we ask three times to enter the student name, ID, and age, at the end, what are we going to do? What do we need to do? Have a look here. What is the usage of this line? Well done. We are creating an **object instance** of the class. The name of our class is `Student`. Don't get confused because we have many names based on "student". Let's go up here. The name of the class is `Student`. If you define something else, for example, a `Student_Underlying_Class`, then here, if you want to have access, you need to define the instance as `student_underlying_class`. Then you define just one instance or one object and use that object (`student_this`) as an instance to call one of the methods inside the class. Which method was that? `set_data`. It means you want to set the name, age, and student ID. Finally, because you have information for multiple students...

设置初始值和函数目的

我们将为此资产设置初始值。这是强制性的吗？是的，目前是，因为稍后我们将根据初始化器更新它。目前，我们将把它留作一个设置初始值的函数。

你可能以另一种方式编写；它可能仍然是正确的。我们不应该都遵循完全相同的风格；编码风格可以有所不同，但所有实现都应该是正确的。

DISPLAY_INFO 函数

我们有另一个函数用于显示信息。我们将显示什么作为输出？只有两个值：一个是**姓名**，另一个是**年龄**。对于输出，我们不关心学生ID。这就是为什么我们只返回姓名和年龄，在我们调用 `display_info` 函数的地方。

从 MAIN 调用函数

通过 `main` 函数（大约在第55行编写），我们需要调用的第一个函数是 `collect_students`。对于Python编译器，我们可以开始调用这个函数。让我们跳转到这个函数。

COLLECT_STUDENTS 函数内部

从第55行，我们转到第20行。在第20行，我们有 `collect_students` 函数。让我们打开这个函数查看内部。名称很明确：我们想要收集多少名学生的输入数据。如果你提到三，你可以将其更改为任意数量。你甚至可以询问最终用户他们想要输入多少名学生。

理解 FOR 循环

这个 `for` 循环与 C++ 或 C 编程中的循环，甚至 MATLAB 或 Java 中的循环有点不同。这是 Python 风格的 `for` 循环。`i` 是循环的计数器。它默认自动从零开始；它是一个索引（0, 1, 2）。在 `range(3)` 中，意味着将考虑 0、1 和 2，但不会考虑 3。

我们要求打印"学生"，因为这是第一个学生、第二个学生、第三个学生。这就是为什么我们使用 `i+1`。我们没有学生0；我们有学生1号、学生2号、学生3号。这就是我们想要打印它的原因。然后我们询问最终用户："姓名是什么？" 你记得昨天我们使用 `input` 函数来询问学生姓名，该值将被存储。所有这三个值（姓名、ID、年龄）都在循环内部收集，因为我们想多次询问它们。

变量作用域和对象创建

👤 学生：这是全局变量还是局部变量？ 🗣️ 讲师：它是局部变量，因为它在函数内部。所有这些变量都是局部的。

当我们要求输入三次学生姓名、ID和年龄后，最后我们要做什么？我们需要做什么？看这里。这一行的用途是什么？很好。我们正在创建类的一个**对象实例**。我们的类名是 `Student`。不要混淆，因为我们有很多基于"student"的名称。让我们看上面这里。类的名称是 `Student`。如果你定义了其他东西，例如 `Student_Underlying_Class`，那么在这里，如果你想访问，你需要将实例定义为 `student_underlying_class`。然后你只定义一个实例或一个对象，并使用该对象（`student_this`）作为实例来调用类内部的一个方法。那是哪个方法？`set_data`。这意味着你想要设置姓名、年龄和学生ID。最后，因为你有多名学生的信息...

PRACTICAL APPLICATION OF LISTS AND DATA STRUCTURES

If you define a `Student_Underlying_Class`, to access it, you need to instantiate it as `student_underlying_class`. You then define one instance or object and use that object (`student_this`) to call a method inside the class, which is `set_data`. This method sets the name, age, and student ID.

Since you have information for three students, the `append` method helps you stack the records on top of each other. The first record contains a student's name, age, and ID. You then append the next student's name, age, and ID, and so on. The order depends on how you collect the data.

👤 **Speaker 4:** What is the data type of this variable? 🗣️ **Mohammad:** Please check it later, as we will work with the `DataFrame` data structure soon. We need to understand the usage of a **list**, when to use it practically, and when to use a `DataFrame`.

In a list, you can store different data types like strings, integers, and floats. We use `append` to keep all information for the three students within a single list, stacking the records.

CODE REVIEW AND FUNCTION PURPOSE

👤 **Mohammad:** This `print`, is it inside the function? What is its usage? What is the application? 🙄 ♂ **Speaker 4:** It prints the data. 👤 **Mohammad:** If we print the data, we should see something here... in that box. 🙄 ♂ **Speaker 2:** It's empty. 👤 **Mohammad:** Oh, it's empty. We are not passing anything. What is the usage of this type? I tried to write it with a different concept so you can learn many new things from this example.

Usually, you want a space in the output for readability, so the results aren't all combined and messy. That's why you add a line space. I will share this code for you to run later. However, you should follow the structure of the coding you've done so far if you haven't finished, as that is correct.

IMPORTANCE OF CODING STYLES AND PROGRAM FLOW

You need to learn from your peers and the different coding styles I share. In the future, when you work in industry and join a company project, you'll need to read and understand existing code. Therefore, familiarity with different coding styles is essential.

The first function collects the input information: name, age, and student ID. The next function should display the students based on age order. After collecting information in the `main` function, the compiler's cursor returns to the calling line. The next line is `display`, so the compiler will execute that syntax, requiring a jump from the call to the function's body.

👤 **Mohammad:** What is inside the body of this function? 👤 **Mohammad:** Just as a message, we want to list the student name and age.

列表与数据结构的实际应用

如果你定义了一个 `Student_Underlying_Class`, 要访问它, 你需要将其实例化为 `student_underlying_class`。然后你定义一个实例或对象, 并使用该对象 (`student_this`) 来调用类内部的一个方法, 即 `set_data`。这个方法用于设置姓名、年龄和学生ID。

由于你有三名学生的信息, `append` 方法可以帮助你记录堆叠起来。第一条记录包含学生的姓名、年龄和ID。然后你追加下一个学生的姓名、年龄和ID, 依此类推。顺序取决于你收集数据的方式。

🧑 **Speaker 4:** 这个变量的数据类型是什么？🧑 **Mohammad:** 请稍后检查，因为我们很快会使用 `DataFrame` 数据结构。我们需要理解列表的用途、何时实际使用列表以及何时使用 `DataFrame`。

在列表中，你可以存储不同的数据类型，如字符串、整数和浮点数。我们使用 `append` 是为了将所有三名学生的信息保存在一个列表中，堆叠记录。

代码审查与函数目的

🧑 **Mohammad:** 这个 `print`，它在函数内部吗？它的用途是什么？应用是什么？🧑 **Speaker 4:** 它打印数据。🧑 **Mohammad:** 如果我们打印数据，我们应该在这里看到一些东西...在那个框里。🧑 **Speaker 2:** 它是空的。🧑 **Mohammad:** 哦，是空的。我们没有传递任何东西。这种类型的用途是什么？我尝试用不同的概念来写，这样你可以从这个例子中学到很多新东西。

通常，你希望输出中有空格以提高可读性，这样结果就不会全部挤在一起显得混乱。这就是为什么你要添加一个空行。我稍后会分享这段代码供你运行。但是，如果你还没有完成，你应该遵循你迄今为止所做的编码结构，因为那是正确的。

编码风格与程序流程的重要性

你需要向同伴学习，并学习我分享的不同编码风格。未来，当你在工业界工作并加入公司项目时，你将需要阅读和理解现有的代码。因此，熟悉不同的编码风格至关重要。

第一个函数收集输入信息：姓名、年龄和学生ID。下一个函数应根据年龄顺序显示学生。在 `main` 函数中收集信息后，编译器的光标会返回到调用行。下一行是 `display`，因此编译器将执行该语法，需要从调用处跳转到函数体内部。

🧑 **Mohammad:** 这个函数体内部是什么？🧑 **Mohammad:** 只是一个消息，我们想要列出学生的姓名和年龄。

DISPLAY BASED ON AGE ORDER

After collecting the information in the `main` function, the compiler's cursor returns to the calling line. The next line is `display`, so the compiler will execute that syntax, requiring a jump from the call to the function body.

🧑 **Mohammad:** What is inside the body of this function? 🧑 **Mohammad:** Just a message; we want to list the student name and age. We want it to be sorted, correct?

We are using a **built-in function** named `sort`. You could write a custom function to compare ages and find the minimum, but you can also simply call the built-in function and pass the student list. We have three records in the list that we want to sort.

LEARNING BUILT-IN FUNCTIONS IN PYTHON

How do we learn about this type of built-in function in Python? I will explain the usage, the `key` parameter using **lambda**, and the `stat` parameter. Generally, when you are new to a function, you should consult the official documentation for all built-in functions in detail.

The `sort` function has many parameters, but we are only using two. At a **master's level**, you should know all the available options you can add to a method like `sorted`. Based on different projects, you can use it in a simple way or a full version.

IMPLEMENTING AND DEBUGGING THE SORT

Let's run the code to see how it works. We want to print our outcome. When you sort, you use `sorted(student_variable)`. However, we need to sort based on **age**, not the default. The current code sorts based on the first element (like name) by default.

 **Mohammad**: What is this? (Referring to the sorting output)  **Speaker 6**: Index? 
Mohammad: Is it a counter?  **Speaker 6**: No, no. No index.  **Mohammad**: Student?
Because you have the `for` loop: `for student in sorted(student_list)`.
`sorted(student_list)` is here. We sorted based on what? We want it based on age.
That's why I asked you, is it correct?

You need to modify this part. Through the activity, I asked you to sort based on age, but this code sorts based on the default key. I will leave it for you to change when I share the code. Currently, it sorts based on the first field (e.g., name). For example, if the first name is "Alex" (starting with 'A') and the second is "Matt" (starting with 'M'), the ASCII code for 'A' is lower than 'M', so Alex's information prints first.

Let's run the code and look at the output. When we run it, it initializes and collects information for three students. Student number one is Alex.

按年龄顺序显示

在 `main` 函数中收集信息后，编译器的光标会返回到调用行。下一行是 `display`，因此编译器将执行该语法，需要从调用处跳转到函数体内部。

🧑🏫 **Mohammad**: 这个函数体内部是什么？🧑🏫 **Mohammad**: 只是一个消息；我们想要列出学生的姓名和年龄。我们希望它是排序的，对吗？

我们正在使用一个名为 `sort` 的**内置函数**。你可以编写一个自定义函数来比较年龄并找到最小值，但也可以直接调用内置函数并传入学生列表。我们的列表中有三条记录需要排序。

学习 PYTHON 中的内置函数

我们如何学习 Python 中这类内置函数？我将解释其用法、使用 **lambda** 的 `key` 参数以及 `stat` 参数。通常，当你接触一个新函数时，应该查阅所有内置函数的官方文档以了解细节。

`sort` 函数有许多参数，但我们只使用了两个。在**硕士水平**，你应该知道可以添加到像 `sorted` 这样的方法中的所有可用选项。根据不同的项目，你可以以简单的方式或完整版本来使用它。

实现和调试排序

让我们运行代码看看它是如何工作的。我们想要打印结果。当你排序时，你使用 `sorted(student_variable)`。但是，我们需要基于**年龄**排序，而不是默认方式。当前代码默认基于第一个元素（如姓名）排序。

🧑🏫 **Mohammad**: 这是什么？（指排序输出）🧑🏫 **Speaker 6**: 索引？🧑🏫 **Mohammad**: 是计数器吗？🧑🏫 **Speaker 6**: 不，不是。不是索引。🧑🏫 **Mohammad**: 学生？因为你有 `for` 循环：`for student in sorted(student_list)`。`sorted(student_list)` 在这里。我们基于什么排序？我们希望基于年龄。这就是我问你，它正确吗？

你需要修改这部分。通过这个活动，我要求你基于年龄排序，但这段代码基于默认键排序。我将在分享代码时留给你修改。目前，它基于第一个字段（例如姓名）排序。例如，如果第一个名字是“Alex”（以'A'开头），第二个是“Matt”（以'M'开头），'A'的 ASCII 码低于'M'，所以 Alex 的信息会先打印。

让我们运行代码并查看输出。当我们运行它时，它会初始化并收集三个学生的信息。第一个学生是 Alex。

Code Execution and Bug Analysis

The ASCII code for 'A' is lower than 'M'. Therefore, Alex's information will be printed first, followed by Matt's. Let's run the code and observe.

When executed, the code initializes and collects information for three students. It first prompts for details about student number one, Alex. Then it asks again for the name, age, and student number for the second and third students.

Identifying a Logical Error

The output appears sorted, but there is a discrepancy. The code was intended to sort by **name**, yet the outcome seems to be sorted by **age**. This is a problem. You need to be mentally present in class to trace through the code and understand its flow step by step to ensure it works correctly.

I will share this code, which is about 90% complete. You need to double-check all parts of it. I will also share how to utilize built-in function features, not just for `sorted`, but for others as well.

 Student: Yes, I think there is a bug. We have sorted by name but here the list is sorted by age.  Lecturer: Yes, definitely. There is a bug. We sorted based on name, but the outcome here appears sorted correctly. This is the issue.

Types of Programming Errors

In the worst scenario, you have no syntax error and the answer *seems* correct, but it isn't. This is called a **logical error**. It means there is an issue with the concept behind the code. A **syntax error** is obvious; the compiler will tell you during compilation if the code structure is incorrect.

Sometimes, as in this case, the code feels correct because there is no obvious error, but the result does not meet our expectations. We have three main categories of errors:

1. Syntax Error
2. Logical Error
3. Runtime Error

For example, consider the operation `X / Y`. If `Y` is zero, what happens? It causes an error—specifically, a **runtime error**, which is the third type.

I will share this Python file on Teams. You can check it in detail, review it again to find any bug, and change it. The most important part is that you need to understand the underlying concepts.

代码执行与错误分析

字母'A'的ASCII码低于'M'。因此，Alex的信息会首先被打印，然后是Matt的。让我们运行代码并观察。

当执行时，代码会初始化并收集三个学生的信息。它首先提示输入第一个学生Alex的详细信息。然后它会再次询问第二个和第三个学生的姓名、年龄和学号。

识别逻辑错误

输出看起来是排序的，但存在一个差异。代码的本意是按**姓名**排序，但结果似乎是按**年龄**排序的。这是一个问题。你需要在课堂上保持思维活跃，逐步跟踪代码并理解其流程，以确保它正常工作。

我将分享这段代码，它大约完成了90%。你需要仔细检查它的所有部分。我还将分享如何利用内置函数的特性，不仅限于 `sorted` 函数，也包括其他函数。

 学生：是的，我认为有一个错误。我们按姓名排序了，但这里的列表是按年龄排序的。

 讲师：是的，确实如此。有一个错误。我们基于姓名排序，但这里的结果看起来排序正确。这就是问题所在。

编程错误的类型

在最坏的情况下，代码没有语法错误，答案**看起来**正确，但实际上并非如此。这被称为**逻辑错误**。这意味着代码背后的概念存在问题。**语法错误**是明显的；如果代码结构不正确，编译器会在编译期间告诉你。

有时，就像本例一样，代码感觉是正确的，因为没有明显的错误，但结果不符合我们的预期。错误主要有三类：

1. 语法错误
2. 逻辑错误
3. 运行时错误

例如，考虑操作 `X / Y`。如果 `Y` 为零，会发生什么？它会导致一个错误——具体来说，是一个**运行时错误**，这是第三种类型。

我将把这个Python文件分享到Teams上。你可以详细检查，再次审阅以发现任何错误，并进行修改。最重要的部分是，你需要理解背后的概念。

Professional Software Engineering

Recap: Error Categories

Given the previous discussion, we have three main categories of error:

1. **Syntax errors**
2. **Logical errors**
3. **Runtime errors**

Assignment Review & Submission

I will share this Python file on Teams. You can check it in detail, review it again to find any bugs, and change it. The most important part is that you need to understand the concepts.

Your coding style might be different. For instance, you might not use global variables, or you might define five functions instead of two. I will check your coding style when you push your code to GitHub.

GitHub Setup & Workflow Demonstration

Before sharing the file, I want to double-check that your coding environment (like VS Code) is connected to your GitHub account. I will demonstrate using VS Code. You can install it and connect it to GitHub via the command line or using GitHub Desktop.

For first-time users, GitHub Desktop is easier because it has a graphical user interface (GUI).

DEMONSTRATING A COMMIT AND PUSH

I will open my GitHub Desktop. My local repository has recent updates from yesterday and today that I haven't committed or pushed yet. This means my online GitHub repository is not synced with these local changes.

By default, your GitHub folder is usually in your Documents directory. To sync, I will commit the changes. You can add a description and then commit.

For example, if you are updating a theme for its first version, you could write "Version 0.1" as the description. You can write whatever you want there. If you fix a bug, you can mention

"Version 0.2" and add a short comment about the bug. This helps you remember the reason for updates across different versions and allows you to backtrack to a previous version if needed. After a few weeks, you can recall why you made certain changes.

After committing, I need to push the changes. Pushing updates my GitHub repository online with these local changes.

VERIFICATION AND BENEFITS

I will show you my GitHub to verify the update. You should check this process yourself the first few times to ensure it works correctly.

Look here in the tool. I updated two files locally, and now, one minute after pushing, these same two files are updated online. This means my GitHub is synced with my local computer.

The key benefit is safety: if I lose my laptop, my code is still in a safe place on GitHub. Everything is available there, and I can pull it down later.

 Lecturer: All good?

Next Steps

Let's share the file with you. Please double-check it, read a little more, finalize your coding part, and be ready to share your code in class.

专业软件工程

回顾：错误类别

根据之前的讨论，错误主要有三类：

1. 语法错误
2. 逻辑错误
3. 运行时错误

作业回顾与提交

我将把这个Python文件分享到Teams上。你可以详细检查，再次审阅以发现任何错误，并进行修改。最重要的部分是，你需要理解背后的概念。

你的编码风格可能有所不同。例如，你可能不使用全局变量，或者你可能定义了五个函数而不是两个。当你将代码推送到GitHub时，我会检查你的编码风格。

GitHub设置与工作流程演示

在分享文件之前，我想再次确认你的编码环境（例如VS Code）是否已连接到你的GitHub账户。我将使用VS Code进行演示。你可以安装它并通过命令行或使用GitHub Desktop将其连接到GitHub。

对于首次使用的用户，GitHub Desktop更容易，因为它具有图形用户界面（GUI）。

演示提交与推送操作

我将打开我的GitHub Desktop。我的本地仓库有昨天和今天的最新更新，我尚未提交或推送这些更改。这意味着我在线的GitHub仓库尚未与这些本地更改同步。

默认情况下，你的GitHub文件夹通常在你的文档目录中。要进行同步，我将提交更改。你可以添加描述信息然后提交。

例如，如果你正在更新某个主题的第一个版本，你可以写“版本0.1”作为描述。你可以在那里写任何你想写的内容。如果你修复了一个错误，可以提及“版本0.2”并添加关于该错误的简短注释。这有助于你记住不同版本间更新的原因，并且如果需要，可以回溯到之前的版本。几周后，你还能回忆起为何进行了某些更改。

提交之后，我需要推送更改。推送操作会使用这些本地更改更新我在线的GitHub仓库。

验证与优势

我将向你展示我的GitHub以验证更新。你自己也应该在最初几次操作时检查这个过程，以确保其正常工作。

看工具这里。我在本地更新了两个文件，现在，推送一分钟后，这两个相同的文件在线更新了。这意味着我的GitHub已与本地计算机同步。

关键优势在于安全性：如果我丢失了笔记本电脑，我的代码仍然安全地存放在GitHub上。所有内容都在那里可用，我以后可以将其拉取下来。

 讲师：都清楚了吗？

后续步骤

让我们把文件分享给你们。请仔细检查，再多读一读，完成你的编码部分，并准备在课堂上分享你的代码。

GitHub Synchronization Confirmation

I have also updated these two files. It means my GitHub is synced with my local computer. The key advantage is safety: if I lose my laptop, my code is in a safe place. All is available on GitHub, and I can pull it later.

 Lecturer: All good?

Activity Instructions and Code Review

Let's share the file with you. Please double-check it, read a little bit more, finalize your coding part, and be ready to share your code in class. It's beneficial for your friends to check and get familiar with different coding approaches.

Process:

1. After you finish your work, open your friends' shared links to understand their solutions.
2. I will put some of the good solutions as sample answers for this activity.

Debugging and File Access

Please double-check the code I shared. If there is any bug I mentioned, you need to update it and add a short inline comment explaining how you fixed it, then reply to the thread. This is a form of **indirect debugging**, which is a basic skill that connects to future testing strategies.

Action Items:

- Download the Python file and run it on your system.
- Take a couple of minutes to debug the shared code.
- If you texted me for access to the directory and couldn't see it, please refresh your page. You should have access now. If not, please text me again.

Final Notes and Python Documentation

I will check your answers randomly. One strong recommendation is to **get used to reading the official Python documentation**. I will share a link later. If you are using a different Python version, note that the latest stable version is 3.14. Version 3.15 is still under development and not final; it's recommended not to use it yet.

GitHub 同步确认

我也更新了这两个文件。这意味着我的 GitHub 已与本地计算机同步。关键优势在于安全性：如果我丢失了笔记本电脑，我的代码存放在安全的地方。所有内容都在 GitHub 上可用，我以后可以将其拉取下来。

🙋 讲师：都清楚了吗？

活动说明与代码审查

让我们把文件分享给你们。请仔细检查，再多读一读，完成你的编码部分，并准备在课堂上分享你的代码。这对你的朋友有好处，他们可以查看并熟悉不同的编码方法。

流程：

1. 完成你的工作后，打开你朋友分享的链接，理解他们的解决方案。
2. 我会将一些好的解决方案作为本次活动的示例答案。

调试与文件访问

请仔细检查我分享的代码。如果我提到了任何错误，你需要更新它，并添加一个简短的内联注释，解释你是如何修复的，然后回复到该主题。这是一种**间接调试**形式，是一项基本技能，与未来的测试策略相关联。

待办事项：

- 下载 Python 文件并在你的系统上运行它。
- 花几分钟时间调试共享的代码。
- 如果你曾发信息给我请求访问目录但看不到它，请刷新你的页面。你现在应该可以访问了。如果还是不行，请再次发信息给我。

最后说明与 Python 文档

我将随机检查你们的答案。一个强烈的建议是习惯阅读官方的 **Python 文档**。我稍后会分享一个链接。如果你使用的是不同的 Python 版本，请注意最新的稳定版本是 3.14。版本 3.15 仍在开发中，并非最终版本；建议暂时不要使用它。

最后说明与 Python 文档

一个强烈的建议是习惯阅读官方的 **Python 文档**。我稍后会直接分享链接。

PYTHON 版本与官方文档

如果你使用的是不同的 Python 版本，请注意最新的稳定版本是 3.14。版本 3.15 仍在开发中，并非最终稳定版本，建议在学习阶段暂时不要使用它，直到它发布稳定版本。

你可以直接访问官方网站 `python.org`，那里有一个文档链接。所有**内置函数**都有针对你使用的特定版本的详细解释。如果你使用的 Python 版本不同，编码风格或功能可能略有差异，这就是为什么官方文档是你最可靠、可以随时查阅的资源。

官方文档的重要性

我稍后会分享电子书，但它可能只在几个月内保持稳定，之后会更新。因此，最好使用官方网站的原始文档，这样你将永远处于安全地带。通常，资深软件开发者除了像这样的原始资源外，不会查阅其他资源。

例如，稍后我们将使用 `numpy` 库或 `pandas` 库。如果你想学习某些内容或代码中出现错误，你需要直接查阅官方文档，以了解如何改进。这不仅解决了错误，还能为你的未来发展学习更多东西。

如何使用 PYTHON 文档查找函数

我将向你们展示如何详细查找 `sorted` 内置函数。要查找任何函数的描述，你只需在左侧选择你正在使用的 Python 版本，然后在搜索栏中输入函数名称。例如，搜索 "sorted" 函数。

点击搜索结果，你可以看到第一行是 `sorted` 函数的一般结构写法及其解释。大多数函数页面在向下滚动时，都会有几个示例。我认为这是学习的最佳来源，它没有限制，不会过时，是你始终可以信赖的资料。

关于类初始化器的讨论

🧑🏫 Mohammad: 使用初始化器是强制性的吗？我们为什么使用它？ 🧑🏫 ♂ Speaker 3: 为了初始化，就像我之前解释的 `self.name`, `self.age`, `self.children` 那样。因此，我定义了初始化器。 🧑🏫 Mohammad: 是为了给变量赋予初始值。那么，是否每次都必须使用初始化器呢？它不是强制性的，但这是一种为变量定义初始值的常见方式。

Final Notes & Python Documentation

One strong recommendation is to **get used to reading the official Python documentation**. I will share the link directly later.

PYTHON VERSION & OFFICIAL DOCUMENTATION

If you are using a different Python version, note that the latest stable version is 3.14. Version 3.15 is still under development and is not a final stable version; it's recommended not to use it until we reach the stable final version, especially while learning.

You can go directly to the official website `python.org`, where there is a link for documentation. All **built-in functions** are explained in detail for the specific version you are using. If you use a different Python version, the coding style or features might be slightly different, which is why the official documentation is the most reliable resource you can check.

IMPORTANCE OF OFFICIAL DOCUMENTATION

I will share an ebook later, but it remains stable only for a couple of months and will be updated later. Therefore, it's better to use the original documentation from the official website; you will be on the safe side forever. Usually, senior software developers will not check any other resources except the original ones like this.

For example, later we will use the `numpy` library or `pandas`. If you want to learn something and encounter an error in your code, you need to go directly to the official documentation to see how you can improve. This not only solves the error but also helps you learn more for your future development.

HOW TO USE PYTHON DOCUMENTATION TO FIND FUNCTIONS

I will show you where the `sorted` built-in function is explained in detail. To find the description for any function, you just need to select the Python version you are using on the left side, then enter the function name in the search bar. For instance, search for the "sorted" function.

Click on the result, and you can see the first line shows the general structure of writing the `sorted` function along with its explanation. For most functions, when you scroll down, there are a couple of examples. I think it's the best source for learning—it's not limited, not outdated, and something you can trust all the time.

DISCUSSION ON CLASS INITIALIZER

 **Mohammad:** Is it mandatory to use the initializer? Why are we using it?  **Speaker 3:** For initializing the ones like `self.name`, `self.age`, `self.children` as I explained earlier. Because of that, I define the initializer.  **Mohammad:** To initiate the initial value to the variable. Is it mandatory all the time to use the initializer? It's not mandatory, but it's a common way to define the initial value for your variable.

Professional Software Engineering - Lecture Transcript Processing

Purpose of Initializers

Mohammad asks why we use initializers. **Speaker 3** explains it's for initializing attributes like `self.name`, `self.age`, and `self.children`. **Mohammad** clarifies that the purpose is to set the initial value for a variable.

 **Mohammad:** Is it mandatory all the time to use the initializer?  **Mohammad:** It's not mandatory, but it's a common way to define the initial value for your variable, if it's needed.

He notes that the shared code didn't use an initializer because it will be covered later. The key point was to check if students understood the concept's purpose.

Variable Scope and Data Types

Mohammad inquires about the use of global variables.

 **Mohammad:** Do you have any global variables?  **Speaker 3:** I do. I also use the local ones which I define like main age and student ID.

The discussion shifts to a specific variable named `student` within the `collect_student_info` function. **Mohammad** asks about its data type on line 10. He emphasizes the importance of understanding data types for all variables, functions, and classes, regardless of whether help is sought from friends, assistants, or public repositories.

Determining Data Type in Python

Speaker 3 asks how to define the data type.

 **Speaker 3:** How can we define it?  **Mohammad:** Use the `type` keyword. We have the keyword `type` and then in the parentheses the name of the variable or the name of the function.

The correct syntax to check a variable's type in Python is:

```
type(variable_name)
```

Code Consistency and Review

The lecturer checks on sorting logic, asking if a student sorted by 'H' (likely a variable name). He compares it to his shared code and asks if the results are the same.

 **Mohammad:** Is it the same result?  **Speaker 5:** No.

Mohammad stresses the need for consistency; code cannot be partially correct. He asks if a student named Hashish ran their code, what the result was, and if they modified the original example (e.g., changing from 3 to 5 students or adding a variable).

He concludes that to investigate further, the code needs to be updated appropriately. The segment ends with an unfinished question: "What is the roo..."

专业软件工程 - 讲座文稿处理

初始化器的目的

Mohammad 询问我们为什么使用初始化器。**Speaker 3** 解释这是为了初始化属性，如 `self.name`、`self.age` 和 `self.children`。**Mohammad** 澄清其目的是为变量设置初始

值。

👤 **Mohammad**: 是否每次都必须使用初始化器？👤 **Mohammad**: 这不是强制性的，但如果需要，这是为变量定义初始值的一种常用方式。

他指出共享的代码没有使用初始化器，因为这将在后续课程中讲解。关键点是检查学生是否理解了这个概念的目的。

变量作用域与数据类型

Mohammad 询问了全局变量的使用情况。

👤 **Mohammad**: 你有使用任何全局变量吗？👤 **Speaker 3**: 我有。我还使用了局部变量，比如我定义的 `main age` 和 `student ID`。

讨论转向 `collect_student_info` 函数中一个名为 `student` 的特定变量。**Mohammad** 询问其第 10 行的数据类型。他强调理解所有变量、函数和类的数据类型的重要性，无论是否从朋友、助教或公共代码库寻求帮助。

在 Python 中确定数据类型

Speaker 3 询问如何定义数据类型。

👤 **Speaker 3**: 我们如何定义它？👤 **Mohammad**: 使用 `type` 关键字。我们有关键字 `type`，然后在括号内放入变量名或函数名。

在 Python 中检查变量类型的正确语法是：

```
type(variable_name)
```

代码一致性与审查

讲师检查了排序逻辑，询问一名学生是否按 'H'（可能是一个变量名）进行排序。他将其与自己共享的代码进行比较，并询问结果是否相同。

👤 **Mohammad**: 结果一样吗？👤 **Speaker 5**: 不一样。

Mohammad 强调了一致性的必要性；代码不能部分正确。他询问一名叫 Hashish 的学生是否运行了他们的代码，结果是什么，以及他们是否修改了原始示例（例如，将学生数量从 3 个

改为 5 个，或添加了一个变量)。

他总结道，为进一步调查，需要适当地更新代码。该片段以一个未完成的问题结束："What is the roo..." (什么是 roo...)

Professional Software Engineering

Code Consistency & Investigation

Mohammad emphasizes that code correctness must be consistent; it cannot be partially correct. He inquires about a student's code execution and modifications.

 **Mohammad:** Did you run the code of Hashish?  **Speaker 5:** Yes.  **Mohammad:** What was the result? Did you change it instead of three students to five students? Did you change or add one more variable?

To investigate further, the code needs to be updated. The core question is: **What is the root cause of the issue?** Simply looking at the code without understanding the underlying concepts is insufficient.

The "Ghost in the Machine" & Future-Proofing

This leads to a common scenario in industry. Developers often report: "The project was working well till yesterday, but today it doesn't work. What happened?" The answer isn't a "ghost" or time change. It's a **"black box" in the code** that wasn't designed with the future in mind.

The problematic mindset is thinking only for today: "My life is happy, it's working well." But when you return tomorrow to add features, you find everything is broken. This mindset is very common and leads to significant stress and problems.

Responsibility of a Software Developer

As a professional, you must be responsible for **future development**, not just today. You need to think: **"How can I develop clean, clear, and maintainable code for future updates?"** This foresight is crucial to avoid recurring issues.

Class Engagement & Collaborative Learning

Mohammad shifts to class logistics, encouraging full participation. He aims for 100% engagement from all students, not just 70%. He is flexible in the first two weeks but expects complete participation from the third week onward. If there are 35 students, he needs 35 answers. No one should be left behind; the group must find solutions together.

The process for tackling problems is collaborative:

1. If a task is simple, do it quickly.
2. If it's difficult, first ask a friend for help. The allocated time is flexible for discussion and peer learning.
3. If you haven't started, then ask the lecturer. The purpose is help, not judgment.

These activities are for learning, not marks. They introduce a **new style of coding**. Even good coders will learn new things daily, as simple examples often embed advanced concepts. For instance, in the current exercise about sorting, most agreed it was by name, yet the result was correct—a subtle point similar to an issue encountered in a real-world big data project on GCP with a food staff team, where a senior developer...

专业软件工程

代码一致性与调查

Mohammad 强调代码正确性必须一致；不能部分正确。他询问一名学生代码执行和修改的情况。

🗣️ **Mohammad**: 你运行 Hashish 的代码了吗？🗣️ **Speaker 5**: 是的。🗣️ **Mohammad**: 结果是什么？你是把三个学生改成了五个学生吗？你更改或添加了更多变量吗？

为了进一步调查，需要更新代码。核心问题是：**问题的根本原因是什么？** 仅仅查看代码而不理解其背后的概念是不够的。

“机器中的幽灵”与面向未来设计

这引出了业界的一个常见场景。开发者经常报告：“项目直到昨天都运行良好，但今天就不工作了。发生了什么？”答案不是“幽灵”或时间变化。这是代码中一个没有为未来设计的“黑盒”。

有问题的思维方式是只考虑今天：“我的生活很幸福，它运行良好。”但当你明天回来添加功能时，你会发现一切都崩溃了。这种心态非常普遍，会导致巨大的压力和问题。

软件工程师的责任

作为一名专业人士，你必须对**未来的开发**负责，而不仅仅是今天。你需要思考：“**我如何能开发出干净、清晰且易于未来维护的代码？**”这种远见对于避免重复出现问题至关重要。

课堂参与与协作学习

Mohammad 转向课堂后勤，鼓励全员参与。他的目标是所有学生100%参与，而不仅仅是70%。他在前两周是灵活的，但从第三周开始期望完全参与。如果有35名学生，他需要35个答案。任何人都不应掉队；小组必须共同寻找解决方案。

处理问题的流程是协作式的：

1. 如果任务简单，请快速完成。
2. 如果任务困难，首先向朋友寻求帮助。分配的时间是灵活的，可用于讨论和同伴学习。
3. 如果你还没有开始，那么再询问讲师。目的是提供帮助，而非评判。

这些活动是为了学习，而不是分数。它们引入了一种**新的编码风格**。即使是优秀的编码员也会每天学习新东西，因为简单的例子常常嵌入高级概念。例如，在当前关于排序的练习中，大多数人都同意是按名称排序，但结果却是正确的——这是一个细微之处，类似于在一个与食品团队合作的真实世界 GCP 大数据项目中遇到的问题，当时一位高级开发人员...

LEARNING FROM REAL-WORLD PROJECT ISSUES

The current exercise, which seems simple, contains **advanced concepts** that are new to some students. For example, in the sorting task, almost everyone agreed it was sorted by name, yet the result was correct. This subtle issue mirrors a problem encountered in a real-world **big data project** on GCP with a food industry team.

In that project, a senior developer completed a task, and the initial result appeared correct. However, I foresaw a future issue because a junior developer had implemented it in a certain way. We proactively raised this concern in a team meeting to clarify that when writing code, we must be **cautious and clear**. Developing in an unclear or mixed manner means another team member will inherit responsibility for future updates, ultimately delaying the project.

TEAM CULTURE AND SHARED RESPONSIBILITY

Such delays have negative consequences for the **entire team**, not just the individual developer. This reflects the **work culture in New Zealand**, which is team-oriented. Success belongs to the whole team, and conversely, project delays or failures are a negative point for all members. Therefore, working in different cultures requires adaptation; here, it's about **teamwork and mutual support**.

Regardless of your role—whether you're a junior with an intern, a mid-level with junior colleagues, or a senior leading the team—you must support all members. Keeping this collaborative world culture in mind will help you grow faster.

CULTIVATING A CURIOUS AND PROACTIVE MINDSET

You don't need to check every shared code snippet out of curiosity about coding style, as we must move forward to save time. This doesn't mean you shouldn't study during the week; on the contrary, **you should study even more**. We are at a level beyond spoon-feeding.

When I share code, you need a **curious mind**. For instance, if a class has only one instance, define a second. Change this, change that, check it, use a debugger watch to inspect variables—ask **what is the data structure** and **why** we use it. This mindset of constantly asking "why" will accelerate your learning. After a few weeks, you'll see significant progress in how you learn.

OBSERVATION ON CODE INITIALIZERS

Let's move forward. I noticed some of you in your shared work are using the **initializer**. I think it's clear from the code I shared—I didn't use it directly, but I used it indirectly.

从现实项目问题中学习

当前的练习看似简单，却包含了一些对学生来说较新的**高级概念**。例如，在排序任务中，几乎所有人都同意它是按名称排序的，但结果却是正确的。这个细微的问题反映了一个在与食品行业团队合作的**GCP大数据项目**中遇到的真实问题。

在那个项目中，一位高级开发人员完成了一项任务，初始结果看起来是正确的。然而，我预见到了未来的一个问题，因为一位初级开发人员以某种方式实现了它。我们在团队会议中主动提出了这个担忧，以阐明在编写代码时，我们必须**谨慎且清晰**。以不清晰或混合的方式开发，意味着其他团队成员将承担未来更新的责任，最终导致项目延迟。

团队文化与共同责任

此类延迟会对**整个团队**产生负面影响，而不仅仅是开发人员个人。这反映了**新西兰的工作文化**，即以团队为导向。成功属于整个团队，反之，项目延迟或失败也是所有成员的负面记录。因此，在不同文化中工作需要适应；在这里，核心是**团队合作与相互支持**。

无论你的角色是什么——是带有实习生的初级人员、带有初级同事的中级人员，还是领导团队的高级人员——你都必须支持所有成员。牢记这种协作的世界文化将帮助你更快地成长。

培养好奇心和主动思维

你不需要出于对编码风格的好奇而去检查每一个共享的代码片段，因为我们必须向前推进以节省时间。这并不意味着你周末不应该学习；恰恰相反，**你应该学习得更多**。我们处于一个超越“填鸭式”教学的层次。

当我分享代码时，你需要一个**好奇的头脑**。例如，如果一个类只有一个实例，那就定义第二个。修改这里，修改那里，检查它，使用调试器监视来检查变量——询问**数据结构是什么以及为什么**我们使用它。这种不断追问“为什么”的思维模式将加速你的学习。几周后，你将看到自己在学习方式上的显著进步。

关于代码初始化器的观察

让我们继续。我注意到你们中一些人在共享的作品中使用了**初始化器**。我认为从我分享的代码中可以看得很清楚——我没有直接使用它，但我是间接使用的。

The "Why" Mindset and Initializer Clarification

Always keep asking "why" in your mind. Why are you developing this way? Why did he develop that way? Compare different approaches. After a couple of weeks, you will see a significant improvement in how you learn.

OBSERVATIONS ON CODE INITIALIZERS

I noticed a couple of you in your shared work are using the **initializer**. From the code I shared, I didn't use it directly, but I used it indirectly. I defined a function to initiate the values. Here, the name of the class is `Color`, and we have the initializer function for `red`, `green`, and `blue`.

This cell (the variable's scope) holds the initial values for red, green, and blue. These are the three variables with their initial values. We have a function. Is it inside the class? No.

👤 Lecturer: What is this `Color`? 👤 Lecturer: What is that again? 👤 Lecturer: Object.
👤 Lecturer: What is this? 🗣️ Student: What is this? 👤 Lecturer: What is this? 👤
Lecturer: The name of the class. Don't get confused because the name is `Color`. Even though the 'C' is uppercase, it's the name of the class.

We then initialize this object with the values `255`, `123`, and `53`. You can imagine it as an RGB pixel (red, green, blue). We initiated it with these initial values. We have one object, and then we want to display it.

We pass this object to the `display` function. You can see `display`. We pass the object and then we print it out.

👤 Lecturer: What is the output? If you run it, what do you see? Again, what we're going to see when you run this code is `255`, `123`, and `53`. Okay? Because you have these values. Am I right or not? Sometimes I'm not telling you the truth because I want to check if you are awake. If you run this code, you're going to see these three numbers, or are you going to see something else? Yes, you need to be confident. Yes, these three numbers. We have the object, and we are calling these three variables `red`, `green`, and `blue`. You initiated them with these values, that's why you're going to get these three numbers.

We learned the **initializer**. In the initializer, because we are going to call that parameter to initiate the value, we can have a second part.

Look here: `red`, `green`, `blue`. Then, have a look. It's exactly the same as the first one. Exactly the same as this outcome. We're going to write the code in a different way again.

👤 Lecturer: `display` – is it inside the class? Oh yes, for this time, yes. Was it inside for the first one? No, it wasn't inside the class. That's right. Now, this cell is accessible. We are using the cell. It's exactly the same thing. The output is the same: these numbers.

I want to teach you the **usage of self** with an example because I don't want you to memorize anything. As a developer, you don't need to memorize.

“为什么”思维模式与初始化器澄清

要始终在脑海中保持追问“为什么”。为什么你要这样开发？他为什么那样开发？比较不同的方法。几周后，你将看到自己的学习方式有显著进步。

关于代码初始化器的观察

我注意到你们中一些人在共享的作品中使用了**初始化器**。从我分享的代码来看，我没有直接使用它，但我是间接使用的。我定义了一个函数来初始化值。这里，类的名称是 `Color`，我们为 `red`、`green` 和 `blue` 准备的初始化器函数。

这个单元（变量的作用域）保存着 `red`、`green` 和 `blue` 的初始值。这是三个带有初始值的变量。我们有一个函数。它在类内部吗？不。

👤 讲师：这个 `Color` 是什么？👤 讲师：那又是什么？👤 讲师：对象。👤 讲师：这是什么？👤 学生：这是什么？👤 讲师：这是什么？👤 讲师：类的名称。不要因为名字是 `Color` 而感到困惑。即使 'C' 是大写，它也是类的名称。

然后我们用值 `255`、`123` 和 `53` 初始化这个对象。你可以把它想象成一个 RGB 像素（红、绿、蓝）。我们用这些初始值初始化了它。我们有一个对象，然后我们想显示它。

我们将这个对象传递给 `display` 函数。你可以看到 `display`。我们传递对象，然后打印出来。

👤 讲师：输出是什么？如果你运行它，你会看到什么？再说一次，当你运行这段代码时，我们将看到 `255`、`123` 和 `53`。对吗？因为你拥有这些值。我说的对吗？有时我并没有告诉你真相，因为我想检查你是否清醒。如果你运行这段代码，你会看到这三个数字，还是会看到其他东西？是的，你需要有信心。是的，就是这三个数字。我们拥有对象，并且我们正在调用这三个变量 `red`、`green` 和 `blue`。你用这些值初始化了它们，这就是为什么你会得到这三个数字。

我们学习了**初始化器**。在初始化器中，因为我们要调用那个参数来初始化值，我们可以有第二部分。

看这里：`red`、`green`、`blue`。然后，看一下。它和第一个完全一样。和这个结果完全一样。我们将再次以不同的方式编写代码。

👤 讲师：`display` – 它在类内部吗？哦，是的，这次是的。第一次它在类内部吗？不，它不在类内部。这就对了。现在，这个单元是可访问的。我们正在使用这个单元。这完全是同一回事。输出是相同的：这些数字。

我想通过一个例子教你 `self` 的用法，因为我不希望你们死记硬背。作为一名开发者，你不需要记忆。

H2: Understanding `self` and Method Invocation

Again, `display` – is it inside the class? Oh, yes, for this time, yes. For the first one, no, it wasn't inside the class. That's right. Now, this cell is accessible. We are using the cell. It's exactly the same thing. The output is the same: these numbers.

I want to teach you the **usage of `self`** with the example because I don't want you to memorize anything. As a developer, you don't need to memorize anything. All the coding is just available in one search online. You don't need to memorize, but you need to know the concept behind it and how to use it in different ways.

We again have one object initializer from the class. We assign the initial value and then call the function. Please compare these two slides with each other.

Here, we are passing the object to the function. We define the object and then pass it to the function. If the method belongs to the class, then `self` will take the responsibility. You don't need to pass the object to that function anymore because it belongs to the body of the class. It's a different style of coding.

H2: Project Structure and Code Simplicity

 Student: Can we define more than one class in the project?  Lecturer: Yes. It's the same as how many functions we can define. As many as you want. But don't use too many because project management is the most important thing. You need to optimize your code. Otherwise, you will get dizzy, especially after a couple of weeks, you cannot remember how you developed it. Don't make it complex for yourself.

Usually, some of my students and even colleagues try to do the coding in a really complex way. I don't believe it's a positive point. If you write code that's not easy to understand, you will be the first person to face the issue in the future. Later, you will go back to it and take more minutes to understand or remember. Sometimes you cannot remember.

For example, one of my team members and I wanted to update a project feature. I asked that senior developer to update it because the client requested it. He asked me, "Did I develop this part?" Even he couldn't remember because of the overly complex style. I told him you shouldn't write code in a really complex way. It doesn't show you are senior. Senior people are usually humble; they try to support and write code in an efficient and simple way so everyone on the team can understand, even if they are not well-experienced.

H2: Reviewing Methods

So, how about the method? I think we discussed methods a lot; it's not a new thing. Do I need to explain here what is the name of the method? What is it?

H2: 理解 `self` 与方法调用

再次，`display`——它在类内部吗？哦，是的，这次是的。第一次，不，它不在类内部。这就对了。现在，这个单元是可访问的。我们正在使用这个单元。这完全是同一回事。输出是相同的：这些数字。

我想通过一个例子教你 `self` 的用法，因为我不希望你们死记硬背。作为一名开发者，你不需要记忆任何东西。所有的代码只需一次在线搜索即可获得。你不需要记忆，但你需要知道其背后的概念以及如何以不同方式使用它。

我们再次从类中创建一个对象初始化器。我们分配初始值，然后调用函数。请将这两张幻灯片相互比较一下。

这里，我们将对象传递给函数。我们定义对象，然后将其传递给函数。如果方法属于类，那么 `self` 将承担这个责任。你不再需要将对象传递给那个函数，因为它属于类的主体。这是一种不同的编码风格。

H2: 项目结构与代码简洁性

 学生：我们可以在项目中定义多个类吗？ 讲师：可以。这和我们能定义多少个函数是一样的。想定义多少就定义多少。但不要用太多，因为项目管理是最重要的事情。你需要优化你的代码。否则，你会感到头晕，尤其是几周后，你可能会记不清自己是如何开发的。不要让它对你来说变得复杂。

通常，我的一些学生甚至同事试图以非常复杂的方式进行编码。我不认为这是一个优点。如果你写的代码不易理解，你将是未来第一个遇到问题的人。之后，你会回过头来，花更多时间去理解或回忆。有时你根本记不起来。

例如，我的一个团队成员和我想要更新一个项目功能。我请那位高级开发人员更新它，因为客户提出了要求。他问我：“这是我开发的部分吗？”他甚至因为过于复杂的风格而记不起来了。我告诉他，你不应该以非常复杂的方式编写代码。这并不能显示你是高级开发者。高级人士通常很谦逊；他们努力支持团队并以高效、简单的方式编写代码，这样团队中的每个人都能理解，即使他们经验不足。

H2: 回顾方法

那么，方法呢？我想我们已经讨论了很多关于方法的内容；这不是一个新东西。我需要在这里解释方法的名称是什么吗？它是什么？

H2: Reviewing Methods

Senior developers are usually humble; they try to support the team and write code in an *efficient and simple way*. Everyone on the team can understand it, even if they are not well-experienced.

Now, regarding **methods**, we have discussed them a lot; it's not a new concept. Do I need to explain here what the name of the method is? What is the name of the object? Please compare these two to understand what the difference is between a **function** and a **method**.

 Student: Are you talking about this `find_character` or something else?  Lecturer: What is the name of this `find_character`? It looks like a function, but it's inside a class. That's why the name is a **method**.

I asked a team member with over 20 years of experience, who didn't know the difference, to add a method for a feature. He wrote a function. It worked, but it showed a lack of knowledge. This didn't happen just once. Therefore, during a general professional development meeting, I recommended all team members take a short course.

In all good companies, or at least not in startups, you usually have **professional development**. Each year, you need to study and learn something new. Most of the time, your line manager will recommend courses. You can study more, but their recommendation to take a course, learning module, or exam is mandatory.

H2: Code Example: Function vs. Method

Let's review this together. We have one class here with an initializer. We initiate it using `self.text`. We have one... what is the name of these? Where is the method? Is there any method here?

```
# Example structure
class ExampleClass:
    def __init__(self, text):
        self.text = text

    def find_character(self): # This is a method
        # ... logic
```

We have one function and one method. The name is the same, but the style of coding is different. If you run this code, the result should be exactly the same as the function version.

H2: Activity Announcement & Built-in Methods

For an activity next week, I will write and share it after class to save time. I want to ensure you understand the difference between **functions** and **methods**. The activity will involve using specific methods and functions correctly, just for review and recall.

There are several **built-in methods** available, like `.capitalize()` (which makes the first character uppercase) and `.lower()` (which changes all characters to lowercase). How many other built-in methods are available in Python? And the life...

H2: 回顾方法

高级开发者通常很谦逊；他们努力支持团队并以高效、简单的方式编写代码。团队中的每个人都能理解，即使他们经验不足。

现在，关于方法，我们已经讨论了很多；这不是一个新概念。我需要在这里解释方法的名称是什么吗？对象的名称是什么？请比较这两者，以理解函数和方法之间的区别。

🧑 学生：你是在说这个 `find_character` 还是别的什么？🧑 讲师：这个 `find_character` 叫什么名字？它看起来像一个函数，但它在类内部。这就是为什么它被称为方法。

我曾让一位有超过20年经验但不知道区别的团队成员为一个功能添加一个方法。他写了一个函数。它能工作，但这显示出知识的缺乏。这种情况不止发生了一次。因此，在一次通用的专业发展会议上，我建议所有团队成员参加一个短期课程。

在所有优秀的公司，或者至少不是在初创公司，通常都有专业发展计划。每年，你都需要学习新东西。大多数时候，你的直属经理会推荐课程。你可以学习更多，但他们推荐参加课程、学习模块或考试是强制性的。

H2: 代码示例：函数 vs. 方法

让我们一起回顾一下。这里有一个类和一个初始化器。我们使用 `self.text` 来初始化它。我们有一个.....这些叫什么名字？方法在哪里？这里有方法吗？

```
# 示例结构
class ExampleClass:
    def __init__(self, text):
        self.text = text
```



```
def find_character(self): # 这是一个方法
    # ... 逻辑
```

我们有一个函数和一个方法。名称相同，但编码风格不同。如果你运行这段代码，结果应该与函数版本完全相同。

H2: 活动通知与内置方法

下周有一个活动，我将在课后编写并分享以节省时间。我想确保你们理解**函数**和**方法**之间的区别。该活动将涉及正确使用特定的方法和函数，仅供复习和回顾。

有许多**内置方法**可用，例如 `.capitalize()`（使第一个字符大写）和 `.lower()`（将所有字符改为小写）。Python 中还有多少其他内置方法可用？以及生命周期...

H2: Python 内置方法与语言优势

确保你以正确的方式使用它。这只是为了复习和回顾。你稍后会读到它。

有许多**内置方法**可用，例如 `.capitalize()`（使第一个字符大写）和 `.lower()`（将所有字符改为小写）。Python 中还有多少其他内置方法可用？Python 开发者的生活比 Java 开发者要容易得多。如今，Java 编译器已经由 Java 的编译器开发者改进了很多，但 Python 仍然要容易得多。

如果你正在学习新的东西，你也会有这种感觉。如果你有其他编程语言的经验，跟上 Python 的步伐对你来说相当容易。它非常强大，不仅适用于数据分析、AI 项目，也适用于所有的应用程序开发。

你可以将 Python 与 UI 集成，用于移动应用。我们将使用 Django、Flask 等作为 UI，一些 SQL 内置数据库，其中一些你需要运行服务器。但所有东西都应该与 Python 集成，如果你有兴趣或有背景，还可以加上其他语言。

你仍然可以结合多种编程语言来完成你自己的项目，这不是针对第一次评估，而是针对第二次。好吧，我想你们会学习这些东西，我相信这对你们来说并不复杂。

处理字符串等等，是我分享的链接里的内容，但现在不是。我分享了 3.6 的更新链接，Python 更新得相当快。条件语句，我认为不是我应该详细讨论的内容，大于等于不是... 这不是... 如果你忘记了，可以快速阅读它，或者它是一个逻辑...

👤 Lecturer: 我需要解释这个表格吗？如果你不知道，不要感到害羞。请阅读它。如果你感到害羞，问我或者问我。我每周一或周二有空，取决于那一周的时间安排。你可以预约时间，可以给我发信息，也可以来和我面对面交流，如果你觉得更舒服的话，如果你有任何问题。

好的。这个操作符，双星号。False, true。

👤 Lecturer: 我们需要将 false 的第一个字母大写吗？这是强制性的吗？还是 true？或者我们可以把它改成小写？再次确认，你们都同意它应该首字母大写，还是我可以写成小写？大写是强制性的吗？是的，这个简单的事情你需要测试一下，或者不是 100% 在 Python 中显示，请测试一下。你有 Google Colab，你有 Jupyter Notebook，你有 VS Code，不同的 IDE，可以用来测试这些小东西。

无论如何，我会留给你们。我只是要分享一个活动。让我们快速互相核对一下出勤情况。但是，与此同时，我正在检查出勤。有什么问题吗？当你们没有问题的時候，意味着我要分享的活动应该很容易。然后你们所有人将要工作并分享结果。

H2: Python 内置方法与语言优势

确保你以正确的方式使用它。这只是为了复习和回顾。你稍后会读到它。

有许多内置方法可用，例如 `.capitalize()`（使第一个字符大写）和 `.lower()`（将所有字符改为小写）。Python 中还有多少其他内置方法可用？Python 开发者的生活比 Java 开发者要容易得多。如今，Java 编译器已经由 Java 的编译器开发者改进了很多，但 Python 仍然要容易得多。

如果你正在学习新的东西，你也会有这种感觉。如果你有其他编程语言的经验，跟上 Python 的步伐对你来说相当容易。它非常强大，不仅适用于数据分析、AI 项目，也适用于所有的应用程序开发。

你可以将 Python 与 UI 集成，用于移动应用。我们将使用 Django、Flask 等作为 UI，一些 SQL 内置数据库，其中一些你需要运行服务器。但所有东西都应该与 Python 集成，如果你有兴趣或有背景，还可以加上其他语言。

你仍然可以结合多种编程语言来完成你自己的项目，这不是针对第一次评估，而是针对第二次。好吧，我想你们会学习这些东西，我相信这对你们来说并不复杂。

处理字符串等等，是我分享的链接里的内容，但现在不是。我分享了 3.6 的更新链接，Python 更新得相当快。条件语句，我认为不是我应该详细讨论的内容，大于等于不是... 这不是... 如果你忘记了，可以快速阅读它，或者它是一个逻辑...

👤 Lecturer: 我需要解释这个表格吗？如果你不知道，不要感到害羞。请阅读它。如果你感到害羞，问我或者问我。我每周一或周二有空，取决于那一周的时间安排。你可以预约时间，可以给我发信息，也可以来和我面对面交流，如果你觉得更舒服的话，如果你有任何问题。

好的。这个操作符，双星号。False, true。

👤 Lecturer: 我们需要将 false 的第一个字母大写吗？这是强制性的吗？还是 true？或者我们可以把它改成小写？再次确认，你们都同意它应该首字母大写，还是我可以写成小写？大写是强制性的吗？是的，这个简单的事情你需要测试一下，或者不是 100% 在 Python 中显示，请测试一下。你有 Google Colab，你有 Jupyter Notebook，你有 VS Code，不同的 IDE，可以用来测试这些小东西。

无论如何，我会留给你们。我只是要分享一个活动。让我们快速互相核对一下出勤情况。但是，与此同时，我正在检查出勤。有什么问题吗？当你们没有问题的時候，意味着我要分享的活动应该很容易。然后你们所有人将要工作并分享结果。

Attendance Check and Activity Introduction

I will leave the previous topic for you to explore. I will now share an activity. Let's quickly check attendance with each other. Meanwhile, I am checking the attendance list. Any questions? When you have no questions, it means the activity I'm going to share should be easy. Then all of you will work on it and share the results.

I will mark everyone as present here for now, and then we can proceed. Is the attendance list readable? Okay. Let's help me check the attendance.

First one, what's your name? "I'm Richard." Thank you. Yes, casual was here somewhere. Okay. Next one. Okay, friends next to each other. What's your name? Can you pronounce your name? "Crenjo." Okay, thank you. Next one: "Subjit Singh." Are you in the class? No? Okay.

How about the next? I'm turning. Who is the next? The last name is Adia. Not here today? How about the next? Yes. Yes. I'm supposed to... Yes. "Dimo." Nice present. "Darling." Any questions?

For all the students I've called so far, life is going well? You know what you are doing? Doing fine? Is the concept okay for you? Correct. Very... Bad? Where are you? Ah? "Jin Pio?" Is that correct? What's your name?

 **Speaker 6:** Very close. Okay. It's okay.

The last name is Jin?

 **Speaker 6:** Jin Pio Geng.

Jin Pio Geng. Okay, thank you.

Alright. The next... Well, okay. How about you? Is that okay, the concept or the Python coding? Please text me if you need any help. Penguin? No? Yes? We are talking about Sue. You are... I'm talking about...

Okay. We have another, "Ravindu salon"? Yeah. Is it okay, the concept that we discussed today? Yeah. "Roshan"? Third year Roshan, yeah. "Sandalini"? Is that correct by pronunciation? "Sandal"?

 **Speaker 5:** Yes, sir.

Who is the next? "Sarita"? "Sean"? Yes sir.

 **Speaker 2:** Thank you.

She runs. Say yes, Abdullah. We... Thank you. We...

考勤检查与活动介绍

之前的主题留给你们自行探索。我现在要分享一个活动。让我们快速互相核对一下出勤情况。与此同时，我正在查看考勤名单。有什么问题吗？当你们没有问题的时候，意味着我要分享的活动应该很容易。然后你们所有人都要完成它并分享结果。

我现在先把所有人都标记为出席，然后我们就可以继续了。考勤名单看得清吗？好的。请大家帮我核对一下出勤。

第一个，你叫什么名字？"我是Richard。" 谢谢。是的，casual刚才在的。好的。下一个。好的，旁边的朋友。你叫什么名字？能说一下你的名字吗？"Crenjo。" 好的，谢谢。下一个："Subjit Singh。" 你在课堂上吗？不在？好的。

下一个呢？我转一下。下一个是谁？姓氏是Adia。今天不在吗？下一个呢？是的。是的。我应该... 是的。"Dimo。" 很好，出席。"Darling。" 有什么问题吗？

对于我点到名的所有学生，一切顺利吗？你们清楚自己在做什么吗？还好吗？今天讨论的概念对你们来说可以吗？正确。非常... 糟糕？你在哪里？啊？"Jin Pio?" 对吗？你叫什么名字？

 **学生6:** 很接近了。好的。没关系。

姓氏是Jin吗？

 **学生6:** Jin Pio Geng。

Jin Pio Geng。好的，谢谢。

好的。下一个... 嗯，好的。你呢？概念或者Python编程可以吗？如果需要任何帮助，请给我发信息。Penguin？不？是的？我们在谈论Sue。你是... 我在谈论...

好的。我们还有一位，"Ravindu salon"？是的。我们今天讨论的概念可以吗？是的。"Roshan"？大三的Roshan，是的。"Sandalini"？发音对吗？"Sandal"？

 **学生5:** 是的，老师。

下一个是谁？"Sarita"？"Sean"？是的老师。

 **学生2:** 谢谢。

她跑了。说"是"，Abdullah。我们... 谢谢。我们...

LECTURE TRANSCRIPT PROCESSING

Mohammad continues the roll call. He confirms the presence of Roshan (third year) and Sandalini.

 **Student 5:** Yes, sir.

Mohammad asks who is next, listing names like Sarita and Sean. A student thanks him. He notes someone leaving ("She runs") and prompts Abdullah to respond.

The roll call proceeds with several brief, sometimes unclear interjections ("We", "She", "No", "Yes"). **Mohammad** asks for a student's last name and then their first name.

He then calls for "Winchy", confirms it, and moves on. He asks another student, whose last name might be Leon, for their first name, which is given as "Xion".

 **Student 6:** Yes, sir.

Mohammad asks for this student's first name, which is "Xionin". He notes there are four more students to go and quickly calls out names: San, Xiong Tong, Yasis, and Chen... Yilon.

A student interjects with an unclear phrase in another language ("attività sono a te le classi").

SUMMARY & RECOMMENDED RESOURCES

Admin Summary: This segment is a continuation of the student roll call. The lecturer confirms attendance, clarifies name pronunciations, and manages the logistics of checking in a large group of students.

Academic Summary: No academic concepts related to software engineering were discussed in this transcript chunk. The content is purely administrative.

Recommended Level 9 Resources: As this segment contained no academic content, no specific technical resources are recommended. For the core course material on **Professional Software Engineering**, students should refer to the seminal papers and official documentation related to:

- **Software Process Models** (e.g., Agile Manifesto, Spiral Model papers)
 - **Requirements Engineering**
 - **Software Architecture & Design Patterns** (Gang of Four book, IEEE standards)
 - **Software Quality Assurance & Testing** (ISO/IEC 25010 standard, research on TDD/BDD)
-

完整中文翻译

讲座转录处理

Mohammad 继续点名。他确认了 Roshan（大三学生）和 Sandalini 的出席。



Mohammad 询问下一个是谁，列出了 Sarita 和 Sean 等名字。一名学生向他道谢。他注意到有人离开 ("她跑了")，并提示 Abdullah 回应。

点名在几个简短、有时不清晰的插话（“我们”、“她”、“不”、“是”）中继续进行。**Mohammad** 询问一名学生的姓氏，然后是名字。

接着，他点名 "Winchy"，确认后继续。他询问另一名可能姓 Leon 的学生，其名字是 "Xion"。



Mohammad 询问这名学生的名字，回答是 "Xionin"。他指出还有四名学生，并快速点出以下名字：San、Xiong Tong、Yasis 和 Chen... Yilon。

一名学生用其他语言插了一句不清晰的话 ("attività sono a te le classi")。

总结与推荐资源

行政事务总结：此片段是学生点名的延续。讲师确认出勤情况，澄清姓名发音，并管理大批学生签到的事务性工作。

学术内容总结： 本转录片段未讨论与软件工程相关的学术概念。内容纯属行政事务。

推荐 Level 9 资源： 由于此片段不含学术内容，因此不推荐具体技术资源。关于专业软件工程的核心课程材料，学生应参考相关 seminal papers 和官方文档，例如：

- 软件过程模型（例如，敏捷宣言、螺旋模型论文）
- 需求工程
- 软件架构与设计模式（四人帮著作，IEEE 标准）
- 软件质量保证与测试（ISO/IEC 25010 标准，关于 TDD/BDD 的研究）